

Philips Hue V2 connection for JUNG Visu Pro

Setting up the Philips Hue V2 connection in JVP - step by step

This connection uses the Hue bridge's new **CLIP v2 API** as well as three new JVP features:

- **SSE eventstream**: state changes (switching, dimming, sensor events, battery level, ...) arrive from the bridge in real time, instead of JVP having to keep asking for them. There is therefore no polling interval anymore, unlike the old CLIP v1 connection – **Refresh states** is now only needed for the rare case that the bridge inventory itself changed (see chapter 7).
- **One script for German and English**: `InterfaceDescription.xml` holds both languages in a single file; unlike the old Hue connection there are no longer two separate folders for German/English.
- **E:SetFolderName**: the name reported by the bridge is now also written directly into the instance's folder name – the tree in the editor shows self-explanatory names on its own, without having to rename every instance by hand.

1. Important difference from the old Hue connection: HTTPS is now mandatory

The bridge's CLIP v2 API is **only reachable over HTTPS with a self-signed certificate** – this now applies to the entire communication of this connection (not just the optional presence query as with the old CLIP v1 connection). If the JVP HTTP resource does not accept the bridge's certificate, the connection stays disconnected (see troubleshooting, chapter 10) – when in doubt, first test with **Query presence (geofencing)** in the old connection whether HTTPS to the bridge works at all.

2. Find the Hue bridge IP address

- In the Philips Hue app: **Settings → Hue Bridges → [select bridge] → IP address**.
- Alternatively, look for the device „Philips-hue“ in your router's interface.
- Note this IP address (e.g. 192.168.1.20).

3. Load the interface in JVP

- Open the interface editor and (re)load the **Philips Hue V2** connection (folder `Interfaces\HueV2Interface`).
- If the connection was already open: reload the editor so the current `InterfaceDescription.xml` is picked up.

4. Enter the bridge IP

- Navigate to **Configuration** in the tree.
- Set **Bridge IP address** to the IP noted in step 2.
- Optional: enable **Auto-update bridge IP** (cloud service `discovery.meethue.com` , requires internet access – see chapter 9).

5. Perform pairing (get the API key)

1. Set **Start pairing** to 1 .
2. Within 5 minutes, press the **link button** on the Hue bridge (the round button on top of the device).
3. Watch **Pairing state** – it switches to “Paired” once it worked. Right after that, an initial full sync and the eventstream start automatically.
4. **API key (username)** is then filled in automatically – this value is also used as the `hue-application-key` header for all CLIP v2 requests and the eventstream.

If “Timeout” appears: repeat step 5 and press the link button a little more promptly.

6. Find the Hue IDs of your devices

The easiest way is directly through the interface itself:

1. Set **List inventory** (Configuration) to 1 .
2. Under **Status** there are now five dedicated fields – **Inventory lights**, **Inventory rooms/zones**, **Inventory**

scenes, Inventory sensors, Inventory automations – each a tab-separated, name-sorted list with the v2 resource ID and name.

3. Copy the content – it can be pasted directly into Excel/a spreadsheet (tabs = column separators) or used as a cheat sheet when creating the instances (step 7).

CLIP v2 IDs are long UUIDs (e.g. `a1b2c3d4-...`) instead of the short numbers from CLIP v1 – easiest to copy from the inventory rather than typing them by hand.

Alternative – auto-assign IDs: instead of copying every ID individually, you can simply create the required instances (empty, without an ID) in the desired order. Set **Auto-assign IDs** (Configuration) to `1`: the resources found are sorted by name per category and assigned in order to the instances already created. Fields that already contain an ID are left untouched.

7. Create instances in the JVP editor

Each category has its own main folder. Inside it, create a new instance for every device you want to control (right-click the subfolder → new instance):

Main folder	Subfolder (instance)	Instance for	ID field to fill in
Lights	Light	every individual light	Hue light ID
Rooms/Zones	Group	every room or zone	Hue room/zone ID
Scenes	Scene	every scene	Hue scene ID
Sensors	Sensor	every motion/temperature/light-level sensor, remote/switch, contact sensor	Hue sensor ID
Automations	Automation	every Hue automation	Hue automation ID

As soon as an ID is entered, the interface immediately loads that instance's state once in full (no need to wait for the next eventstream cycle).

Note on sensors: unlike CLIP v1, motion, temperature and light-level sensors are **three separate resources** in CLIP v2 – a single physical Hue motion sensor, for example, appears three times in the inventory (motion/temperature/light_level), each with its own ID. Create a separate sensor instance with the relevant ID for each measurement you want. Name and battery level are taken automatically from the owning physical device (sensor resources themselves have no name of their own in CLIP v2):

Device type	Type (SENSOR_TYPE)	Value (text)	Contact data point
Motion sensor	motion	“motion detected” / “no motion”	–
Temperature sensor	temperature	e.g. “21.3 C”	–
Light-level sensor	light_level	e.g. “150 lx”	–
Remote/switch	button	last button event as text (e.g. “short_release”)	–
Window/door contact	contact	“open” / “closed”	Contact (window/door) (1=open, 0=closed)

Note on rooms/zones: for **Hue room/zone ID**, enter the ID of the room or zone itself (not the `grouped_light` service) – the interface resolves the associated `grouped_light` service that actually performs the switching automatically and caches its ID internally in the (not visualization-visible) **Grouped-light service ID** field.

Note on color: hue/saturation are converted internally to the bridge's `xy` color space (CLIP v2 no longer has a native hue°/saturation%). This works well for everyday use, but can cause a hue deviation of up to roughly 15–20° at fully saturated primary colors (pure blue/green/...) compared to the value you set – a known, unavoidable characteristic of the `xy` conversion, not a defect of the interface. For color-critical uses, use the Hue app or a scene instead.

8. Testing

Command and feedback points: every controllable value (on/off, brightness, hue, saturation, color temperature, automation active) exists twice – once as a **command point** without a suffix (write-only) to switch from JVP, and once as a **feedback point** with the “(feedback)” suffix (read-only) for the actual state reported by the bridge.

- Switch a light via **On/Off** and check that **On/Off (feedback)** follows immediately (via the eventstream, not just after the next query).
- Change **Brightness %** and check **Brightness % (feedback)**.

- For color lights: test **Hue angle** / **Saturation %** resp. **Color temperature K** (each with its matching feedback, see the color note above).
- Trigger a scene via **Activate**.
- **Status → Connection** should show “connected” (eventstream is currently receiving data), **Last event** counts up on every bridge change.

If something changes directly on the bridge (e.g. via the Hue app), JVP follows automatically via the **eventstream** – a manual **Refresh states** is not needed for that. It is useful, however: after creating new instances, after **Auto-assign IDs**, or if the connection has been down for a while and you want current values immediately instead of waiting for the next event.

9. Optional: auto-update bridge IP

Hue bridges normally obtain their IP address via DHCP – if it changes (e.g. after a router restart), the interface would otherwise permanently lose the connection until someone enters the new IP manually.

- Set **Auto-update bridge IP** (Configuration) to “on” to avoid this.
- If the eventstream drops or a full sync fails, the interface looks up the current IP via Philips’ official cloud service `discovery.meethue.com` and enters it into **Bridge IP address** automatically.
- Requires internet access for the JVP HTTP resource. A detection attempt is made at most every 5 minutes.
- Without internet access, or for a purely local setup: leave this off and instead fix the bridge IP permanently via a **DHCP reservation in the router**.

10. Optional: presence/geofencing

- Set **Query presence (geofencing)** to “on” to evaluate the home/away state of the Hue app (CLIP v2 `geofence_client` resource, automatically fetched together with the rest of the inventory and kept up to date via the eventstream).
- If **Presence** stays permanently on “nobody home” or errors appear under **Last error**, turn the switch off again – the rest of the functionality is unaffected.

Troubleshooting

Symptom	Cause/solution
Connection stays “disconnected”	Wrong bridge IP, empty API key, network unreachable, or the JVP HTTP resource does not accept the bridge’s self-signed certificate (see chapter 1)
Light/room does not respond	Wrong/empty Hue ID, or the instance folder has not been reloaded
Pairing fails	Link button not pressed in time – repeat step 5
Hue/saturation deviate slightly for strong colors	Expected behavior of the <code>xy</code> conversion for fully saturated primary colors, see chapter 7
SCRIPTNAME error while loading	Old interface version cached in the editor – close and reopen the editor
Bridge IP is not auto-updated despite being enabled	No internet access for the JVP HTTP resource, or <code>discovery.meethue.com</code> unreachable – enter the IP manually instead